Instituicao de Ensino Demo

Curso: Engenharia de Software

8o Semestre

Material de Estudo

Blockchain

---

1. INTRODUCAO

Blockchain e uma tecnologia de registro distribuido que permite transacoes seguras e transparentes sem intermediarios. Originalmente criada para o Bitcoin, a blockchain evoluiu para suportar smart contracts (Ethereum), financas descentralizadas (DeFi), tokens nao-fungiveis (NFTs) e aplicacoes empresariais. Para engenheiros de software, entender blockchain abre portas para uma nova geracao de aplicacoes descentralizadas (dApps) e a Web3.

Ementa: Fundamentos de blockchain. Criptografia aplicada. Consenso distribuido. Bitcoin e criptomoedas. Ethereum e smart contracts. Solidity. DeFi e NFTs. Aplicacoes empresariais. Web3.

2. CONCEITOS FUNDAMENTAIS

A seguir, apresentamos os conceitos-chave desta disciplina com explicacoes didaticas e exemplos praticos para facilitar o aprendizado.

2.1 Como Funciona a Blockchain

Uma blockchain e uma cadeia de blocos onde cada bloco contem: hash do bloco anterior, timestamp, dados (transacoes) e nonce. O hash do bloco anterior cria uma cadeia imutavel — alterar um bloco invalida todos os seguintes. Mecanismos de consenso (Proof of Work, Proof of Stake) garantem que todos os nos concordem sobre o estado da blockchain sem uma autoridade central.

Exemplo:

```
Estrutura de um bloco:
Bloco #42:
hash_anterior: "0x7f3a..."
timestamp: 2024-01-15 10:30:00
transacoes: [
{de: "Alice", para: "Bob", valor: 1.5 ETH},
{de: "Carol", para: "Dave", valor: 0.3 ETH}
]
nonce: 48291
hash: "0x00003b..." (comeca com zeros = PoW)

Alterar transacao no bloco #42 muda seu hash,
invalidando bloco #43, #44, ... toda a cadeia!
```

2.2 Smart Contracts e Solidity

Smart contracts sao programas que executam automaticamente na blockchain quando condicoes pre-definidas sao atendidas. Ethereum e a principal plataforma para smart contracts. Solidity e a linguagem de programacao mais usada. Smart contracts sao imutaveis apos o deploy — bugs nao podem ser corrigidos facilmente, tornando auditoria e testes essenciais.

Exemplo:

```
Smart contract simples em Solidity:
pragma solidity ^0.8.0;

contract Votacao {
mapping(string => uint) public votos;
mapping(address => bool) public jaVotou;

```

```
function votar(string memory candidato) public {
require(!jaVotou[msg.sender], "Ja votou!");
votos[candidato]++;
jaVotou[msg.sender] = true;
}
}
```

3. RESUMO DOS PRINCIPAIS TOPICOS

O conteudo programatico da disciplina esta organizado nas seguintes unidades:

- Unidade 1: Fundamentos de blockchain - historia, conceitos, tipos
- Unidade 2: Criptografia aplicada - hash, assinaturas digitais, Merkle trees
- Unidade 3: Mecanismos de consenso - PoW, PoS, DPoS, PBFT
- Unidade 4: Bitcoin - arquitetura, mineracao, transacoes
- Unidade 5: Ethereum e smart contracts - EVM, gas, Solidity
- Unidade 6: Desenvolvimento de smart contracts com Solidity
- Unidade 7: DeFi, NFTs e tokens (ERC-20, ERC-721)
- Unidade 8: Aplicacoes empresariais - supply chain, identidade, votacao

Cada unidade aborda um aspecto fundamental da disciplina, construindo progressivamente o conhecimento necessario para dominar o tema. Recomenda-se estudar as unidades na ordem apresentada.

4. EXERCICIOS DE FIXACAO

Resolva os exercicios abaixo para verificar seu entendimento dos conceitos apresentados.

Questao 1 (Multipla Escolha)

O que garante a imutabilidade da blockchain?

a) Criptografia simetrica
b) Hash do bloco anterior encadeado
c) Servidor central
d) Backup automatico

Questao 2 (Dissertativa)

Compare Proof of Work (PoW) e Proof of Stake (PoS): vantagens e desvantagens de cada mecanismo de consenso.

Questao 3 (Multipla Escolha)

Qual linguagem e mais usada para escrever smart contracts na Ethereum?

a) Python
b) JavaScript
c) Solidity
d) Rust

Questao 4 (Dissertativa)

Proponha uma aplicacao de blockchain para resolver um problema real (fora de criptomoedas). Descreva a arquitetura.

5. DICAS DE ESTUDO E RECURSOS ADICIONAIS

Para maximizar seu aprendizado nesta disciplina, siga estas recomendacoes:

- Comece entendendo Bitcoin antes de ir para Ethereum e smart contracts.
- Pratique Solidity no Remix IDE (remix.ethereum.org) — e online e gratuito.
- Estude os ataques mais comuns em smart contracts (reentrancy, overflow).
- Explore testnets (Goerli, Sepolia) para testar sem gastar dinheiro real.
- Leia "Mastering Ethereum" para uma compreensao profunda.

Bibliografia Recomendada:

- ANTONOPOULOS, A. Mastering Bitcoin. O'Reilly, 2017.
- ANTONOPOULOS, A.; WOOD, G. Mastering Ethereum. O'Reilly, 2018.
- TAPSCOTT, D. Blockchain Revolution. Senai-SP, 2016.

6. GLOSSARIO DE TERMOS IMPORTANTES

Consulte este glossario sempre que encontrar um termo desconhecido durante seus estudos.

- **Blockchain**: cadeia de blocos com registros imutaveis e distribuidos.
- **Smart contract**: programa que executa automaticamente na blockchain.
- **Solidity**: linguagem de programacao para smart contracts na Ethereum.
- **PoW**: Proof of Work — consenso por poder computacional (mineracao).
- **PoS**: Proof of Stake — consenso por participacao (staking).
- **DeFi**: Financas Descentralizadas — servicos financeiros sem intermediarios.
- **NFT**: Non-Fungible Token — token unico e nao-intercambiavel.
- **Gas**: taxa paga para executar operacoes na Ethereum.
- **dApp**: aplicacao descentralizada que roda em blockchain.